

AgentIA — Informe Final

Del dolor al estado actual: la propuesta, la auto-mejora, el diseño guiado por especificación, la orquestación, la interfaz y el coste

Ingeniería AgentIA

2026-09-29

Contenido

1	El dolor	1
2	La propuesta	1
3	La propuesta de auto-mejora	2
4	Estado actual	2
5	El diseño guiado por especificación	2
6	Orquestación de agentes y LangGraph	3
7	El flujo de la aplicación	3
8	La interfaz de usuario	4
9	El coste	4
9.1	Uso de tokens (actual, del espejo MLflow y del registro)	4
9.2	Datos que faltan y qué hacer	5
10	Conclusión	5

1 El dolor

Generar un microservicio de producción es caro, lento e inconsistente. Cada servicio debe obedecer las mismas reglas — capas estrictas, contratos inmutables, errores centralizados, compilación hermética — y cada incumplimiento se paga en revisión humana, en defectos en producción o en deuda técnica. El problema no es escribir código; es garantizar que el código cumple un estándar, de forma repetible y auditable.

El síntoma que lo hizo tangible. Durante meses la plataforma reportó “**0 de 15 sesiones verificadas**”: no podía demostrar que su propia salida compilaba. La causa resultó ser un defecto de montaje Docker de un solo carácter, no una limitación del generador. Un sistema que no puede verificar su propia salida no puede prometer fiabilidad — y esa era exactamente la brecha de producto.

2 La propuesta

Microservice Code Studio: un agente autónomo que convierte una especificación formal en un microservicio Spring Boot completo, compilado y probado, y que se niega a reportar éxito salvo que

eso sea cierto.

Entrada	una especificación formal (blueprint): entidades, atributos, validaciones, historias BDD, arquitectura en 4 capas
Generación	un agente LangGraph de ocho etapas, gobernado por una compuerta de cumplimiento determinista
Verificación	<code>mvn test -o</code> en un contenedor Docker hermético sin red, con un test de contrato que el modelo no puede autorar
Reparación	auto-reparación acotada (máx. 3 intentos); al límite, BLOQUEADO y pasa a humano
Entrega	descarga ZIP o publicación atómica a Git

La promesa no es “código mejor”; es **gobernanza**: el código cumple las reglas del proyecto, verificado de forma determinista, a un coste predecible.

3 La propuesta de auto-mejora

Sobre el generador se propuso un bucle de auto-mejora reflexiva: un LLM reflexiona sobre los diagnósticos registrados y propone ediciones a un documento de “habilidad”; una edición se admite solo si su contribución medida es positiva, y se desaloja solo si cae por debajo de un suelo de evidencia. Sin pesos, sin GPUs, sin entrenamiento.

La condición que lo gobierna. La literatura y la medición propia coinciden: **un bucle alimentado solo con un veredicto binario no es mejor que ningún bucle** (CoEvoSkills: 41.1 con oráculo opaco frente a 42.4 sin evolución; 71.1 con un verificador de diagnóstico). Por eso la plataforma construyó primero el *canal de diagnóstico* — regla, artefacto, etapa, severidad — y aplazó el optimizador. El bucle (features 014–015) está *construido* pero deliberadamente no se ejecuta.

La medición de polaridad (prohibiciones vs prescripciones) dio **nulo** (1 victoria, 3 derrotas, 2 empates frente al control); en cambio, el *contenido* de las instrucciones importa. La conclusión honesta: antes de cualquier auto-mejora hay que hacer determinista el oráculo de verificación.

4 Estado actual

Cuatro hechos resumen dónde está la plataforma hoy:

1. **Primera compilación verificada.** El defecto de montaje se corrigió y la plataforma ya verifica su propia salida (BUILD SUCCESS, tests ejecutados y contados, no fabricados).
2. **Ambas rutas de producto verifican.** La ruta secuencial generaba código y *nunca lo compilaba*; hoy comparte la misma veta de verificación que el grafo.
3. **La métrica dejó de ser manipulable.** El puntaje pasó de una densidad (mejorable generando más archivos) a una penalización absoluta relativa a una línea base congelada.
4. **El verificador es intermitentemente inestable** ($\approx 33\%$): el mismo espacio de trabajo recompilado da 4/6 bajo carga y 8/8 secuencialmente. Causa probable: reetiquetado SELinux privado (:Z) sobre una caché compartida. Es el siguiente punto a cerrar.

5 El diseño guiado por especificación

El principio rector es *la especificación es la autoridad, no el modelo*:

especificación $\rightarrow$ generación $\rightarrow$ verificación contra la especificación

Especificación	Qué prescribe
Blueprint / <code>spec.md</code> (Spec Kit)	entidades, atributos, validaciones, historias BDD, puerto, capas
Constitución (6 principios)	capas estrictas, contratos inmutables, errores centralizados, offline, quality gates, secretos
Documentos de instrucción (5)	el procedimiento por etapa (contrato, reglas, salida, prohibiciones)
Lista blanca de dependencias	el límite del <code>pom.xml</code>
Contratos (19)	el comportamiento por feature, contra el que se escriben los tests

La generación es una función de la especificación: el modelo propone, la especificación restringe, y el sandbox verifica.

6 Orquestación de agentes y LangGraph

Ocho agentes (nodos) componen la generación; cinco son de modelo, tres son deterministas:

`validador` → `scaffolder` → `dominio` → `servicio` → `controlador` → `test` → `sandbox` ⇌ `reparación`

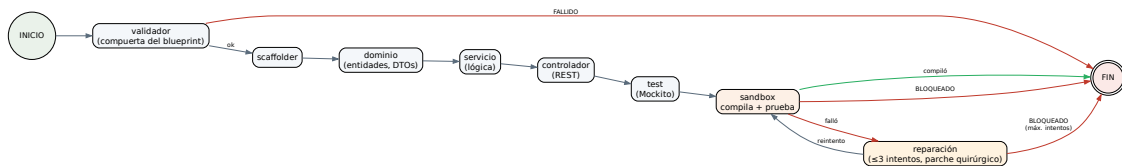


Figura 1: La máquina de estados de generación: columna lineal con un ciclo acotado.

Cada agente de modelo recibe, en orden, *el prefijo de habilidad* (hoy vacío), *el documento de instrucción* (su “system prompt”), *el payload de la tarea* (el slice del blueprint y los artefactos previos), *las rutas que posee* y *el formato de respuesta*. Tres pistas sobre los prompts:

- **Scaffolder:** “no declares ninguna dependencia fuera de la lista blanca” — la regla que hace posible la compilación offline.
- **Dominio:** “los contratos son records nativos; `@Data` está prohibido” — inmutabilidad en el borde.
- **Controlador:** “un `@RestControllerAdvice` que registra la traza” — los errores se centralizan y se diagnostican.
- **Test:** “cada escenario de aceptación se mapea a un método” — la cobertura sigue a la especificación.

7 El flujo de la aplicación

El ciclo de vida completo, de especificación a entrega, en diez etapas:

Fase	Qué ocurre
1. Requisitos	historias BDD y exportación <code>spec.md</code>
2. Arquitectura	catálogo de componentes y contratos REST (Mermaid)
3. Modelos & SQL	entidades JPA, DTOs, <code>schema.sql</code> , diagrama ER
4. Blueprints	ingesta y validación de <code>spec.md</code> / JSON
5. Generación & Monitor	síntesis autónoma + logs en vivo (SSE)
6. Código & Fix	explorador, tests, diffs de reparación, desbloqueo manual
7. Calidad SAST	auditoría, compuerta de calidad, remediación
8. DevOps	Dockerfile, compose, CI/CD, k8s, despliegue local
9. Entrega	ZIP o publicación Git atómica

8 La interfaz de usuario

Una aplicación React/Vite de una sola página, diez pestañas en español, tema claro/oscuro, stepper de ciclo de vida y visores Mermaid para arquitectura y ER. El panel de configuración LLM permite elegir proveedor (Gemini, Groq, OpenAI, DeepSeek o mock offline), clave, modelo y verificar la conexión. La interfaz refleja la verificación honesta de tres estados (verificado / con hallazgos / no evaluable), tras una auditoría que eliminó varios resultados fabricados.

9 El coste

Medido, no estimado, desde el sistema de registro (`cost_tracking.db`) y su espejo MLflow.

Alcance	Llamadas	Coste (USD)	Tokens salida
Experimento de polaridad	750	6.58	—
Cinco congelados	120	0.63	—
Diez difíciles	50	0.64	—
Sondas / sueltas	≈250	≈0.16	—
Total	1219	8.15	12.48M

9.1 Uso de tokens (actual, del espejo MLflow y del registro)

Fuente	Entrada	Salida	Cache	Combinado
Registro (autoritativo)	6.54M	12.48M	2.20M	19.02M
Espejo MLflow	5.49M	9.92M	1.76M	15.41M

Por llamada: ≈5,360 entrada + ≈10,240 salida ≈ 15.6K tokens; por servicio (5 llamadas) ≈78K tokens. La salida es ≈1.9× la entrada (esperado: el generador *emite* código). ≈2.2M tokens de entrada se sirvieron desde caché (≈34%).

Por servicio: **\$0.052–\$0.077** (diez difíciles), ≈\$0.005 por llamada; por etapa, el test es el más caro (\$0.0061) y dominio el más barato (\$0.0012). Todo en `deepseek-flash`.

El espejo MLflow se ha desviado y puede verse vacío. El espejo totaliza **\$6.39** frente a **\$8.15** (subestimación del ≈22%) y **15.41M** tokens frente a **19.02M**; además, `mlflow ui` abre por defecto el almacén vacío `mlruns/` en vez de `mlflow.db`, y los agregados de sesión (168) cayeron en el experimento legado `Default`, con 0 en `agentia`. Hasta reparar el sink, el almacén local es la cifra a citar.

9.2 Datos que faltan y qué hacer

- **El espejo subestima (20–22%) y está partido en dos experimentos.** *Qué hacer:* arreglar el targeting de experimento en `mlflow_sink.py` para que los registros de sesión caigan en `agentia`, y migrar las 877 ejecuciones legadas de `Default` (o aceptar que el espejo es solo indicativo).
- **Falta desglose por modelo.** Todo es `deepseek-flash`; para comparar proveedores se necesita una corrida con otro modelo. *Qué hacer:* una tanda corta (p. ej. Gemini o Groq) sobre el mismo corpus para tener coste y tokens por modelo.
- **El coste de caché es aproximado.** Los tokens cache-hits se registran, pero su base de precio no está verificada contra factura. *Qué hacer:* auditar `pricing_basis` contra la factura del proveedor una vez.
- **Agregados de sesión ausentes en agentia.** *Qué hacer:* confirmar que `mirror_session_cost_record` se invoca al terminar cada sesión y escribe `total_cost_usd` en el experimento correcto.

10 Conclusión

La plataforma pasó de “no puede verificar su propia salida” a “genera, compila y verifica a \$0.06 por servicio, con una selección guiada por verificación que alcanza 100% de cobertura con tres candidatos a \$0.20”. El camino que queda es corto y concreto: **(1)** hacer determinista el verificador (un montaje), **(2)** hacer que la medida pueda moverse (minar fallos de compilación en reglas), y **(3)** solo entonces preguntar si un documento de habilidad aporta algo. El bucle de auto-mejora queda cerrado tras esa compuerta — que es, literalmente, la misma compuerta que el diseño siempre exigió.